

DJANGO DEVELOPER BACKEND TASK

Objective

Build a complete Vehicle Inventory & Booking REST API using Django REST Framework that demonstrates strong backend development skills, proper API structuring, data validation, and the ability to implement business logic within a real-world application scenario. The system should allow users to manage vehicles, create bookings, handle validations, apply pricing logic, and provide clear, well-documented API endpoints suitable for integration with frontend applications or other backend services.

Task Description

1. Project Setup

- Project name: vehicle_system
- App name: inventory

2. Models to Implement

A. Vehicle Model

Fields:

- name (CharField)
- brand (CharField)
- year (IntegerField)
- price_per_day (DecimalField)
- fuel_type (choices: Petrol, Diesel, Electric, Hybrid)
- is_available (BooleanField, default=True)

B. Booking Model

Fields:

- vehicle (ForeignKey → Vehicle)
- customer_name (CharField)
- customer_phone (CharField)
- start_date (DateField)
- end_date (DateField)
- total_amount (DecimalField – auto-calculated)

3. Booking Rules (Real-Life Logic)

- Vehicle cannot be double-booked (no overlapping dates).
- total_amount = days * price_per_day (auto-calc).
- Start date cannot be in the past.
- End date must be after start date.
- Phone number must be 10 digits.
- After booking, vehicle becomes unavailable.

4. API Requirements

Vehicle Endpoints

- GET /api/vehicles/ → List vehicles
- POST /api/vehicles/ → Add new vehicle
- GET /api/vehicles/<id>/ → Vehicle details
- PUT /api/vehicles/<id>/ → Update
- DELETE /api/vehicles/<id>/ → Delete

Booking Endpoints

- GET /api/bookings/ → List bookings
- POST /api/bookings/ → Create booking
- GET /api/bookings/<id>/ → Booking details

Filtering

Support:

- /api/vehicles/?brand=Toyota
- /api/vehicles/?fuel_type=Electric
- /api/vehicles/?is_available=true

5. Deliverables

Candidate must submit a GitHub repository containing:

1. Complete Django Project Code

Models, serializers, views, URLs, and required settings.

2. .env.example File

With sample (dummy) DB and secret key values.

3. README.md

Must include:

- Setup steps
- Installation guide
- Migration commands
- How to run the project
- How to test APIs
- API endpoint list
- Sample JSON for booking

4. Screen Recording (2–5 minutes)

Show:

- Running the project
- API tests in Postman/Swagger
- Booking creation + validation
- Add video file or YouTube/Drive link in README.

5. Host the API on Any Cloud Platform